WEB APPLICATION SECURITY ASSESSMENT REPORT

Target Application
OWASP Juice Shop

Prepared by: Mazin Elsayed

Position: Cybersecurity Intern

Date: 3 April, 2026

Organization: Future Interns

Confidential Security Assessment Report

Table of Contents

1. Executive Summary 3
2. Application Overview 3
3. Scope 3
4. Methodology 3
5. Findings Overview 4
6. Detailed Findings 4
   - 6.1 Lack of Email Verification 4
   - 6.2 Lack of Address Validation 5
   - 6.3 Missing Content Security Policy (CSP)..... 5
   - 6.4 Missing HTTP Strict Transport Security (HSTS) 6
   - 6.5 Insecure Cookie Configuration 6
   - 6.6 Information Disclosure via Headers 7
7. Network Exposure Analysis (Nmap Scan) 8
8. OWASP ZAP Validation 8
9. Recommendations Summary 8
10. Conclusion 8

Executive Summary

This report presents the results of a security assessment conducted on a web application (OWASP Juice Shop).

The objective was to identify common security weaknesses and give practical recommendations to improve the application's security.

Multiple issues were found, including missing security configurations and potential weaknesses in application functionality.

Addressing these issues will help reduce and mitigate security risks and improve user trust.

Application Overview

The application is a web-based e-commerce platform that allows users to browse products, create accounts, and manage a shopping cart.

Users have to authenticate before accessing certain features, such as adding items to the cart and completing purchases. The platform also provides functionality for viewing and managing items within the cart.

Scope

This assessment was conducted on a publicly accessible web application using passive, non-intrusive techniques. No exploitation or harmful actions were performed.

Methodology

The assessment was conducted using:

- Browser Developer Tools
- Nmap (network exposure analysis)
- OWASP ZAP (passive scanning)

Only passive techniques were used.

Findings Overview

#	Finding	Risk
1	No Email Verification	Medium
2	No Address Validation	Medium
3	Missing CSP	Medium
4	Missing HSTS	Medium
5	Insecure Cookies	Medium
6	Info Disclosure	Low

Detailed Findings

Finding 1: Lack of Email Verification During Registration	Medium
Description:	The application allows users to register accounts without verifying ownership of the provided email address. During testing, it was observed that accounts could be created using non-existent or invalid email addresses, and full access to the platform was granted without any verification step.
Impact:	The absence of email verification reduces the reliability and integrity of user accounts. This may lead to: • Creation of fake or automated accounts (spam). • Inability to reliably communicate with users. • Increased risk of fraudulent activities. • Reduced trust in the platform's user base. If left unaddressed, this issue could negatively affect both platform security and business operations.
Recommendation:	It is recommended to implement an email verification mechanism as part of the registration process: • Send a verification link to the provided email address. • Require users to confirm their email before activating the account. • Restrict sensitive actions (e.g., purchases) until verification is completed.

Finding 2: Lack of Address Validation in Checkout Process	Medium
Description:	During testing, it was observed that the application accepts arbitrary and non-existent shipping addresses during the checkout process. Inputs such as random or invalid address data were accepted without any validation or verification.
Impact:	The absence of proper address validation may lead to: • Failed or misdirected deliveries. • Increased operational costs. • Potential abuse through fake or fraudulent orders. • Reduced reliability of order data. This affects both business operations and user trust.
Recommendation:	It is recommended to implement address validation mechanisms, such as: • Input validation for structured address fields. • Integration with address verification services (e.g., postal APIs). • Rejecting clearly invalid or incomplete entries.

Finding 3: Missing Content Security Policy (CSP)	Medium
Description:	The application does not implement a Content Security Policy (CSP) header. CSP is a security mechanism that helps control which resources (such as scripts and styles) are allowed to load in the browser.
Impact:	Without CSP, the application is more vulnerable to client-side attacks such as cross-site scripting (XSS), where malicious scripts can be executed in a user's browser.
Recommendation:	Implement a Content Security Policy header to restrict trusted sources. For example: • Allow scripts only from trusted domains • Block inline scripts where possible

Finding 4: Missing HTTP Strict Transport Security (HSTS)	Medium
Description:	The application does not enforce HTTP Strict Transport Security (HSTS), which ensures that browsers only interact with the server over secure HTTPS connections.
Impact:	Without HSTS, users may be exposed to downgrade attacks or unintentionally connect over insecure HTTP, increasing the risk of data interception.
Recommendation:	• Enable HSTS by adding the following header: "Strict-Transport-Security: max-age=31536000; includeSubDomains"

Finding 5: Insecure Cookie Configuration	Medium
Description:	The application sets cookies without the HttpOnly and Secure attributes. During testing, it was observed that cookies such as cookieconsent_status and language are accessible without these security protections.
Impact:	Without the HttpOnly attribute, cookies can be accessed through client-side scripts, increasing the risk of theft via cross-site scripting (XSS) attacks. Without the Secure attribute, cookies may be transmitted over unencrypted HTTP connections, exposing them to interception by attackers.
Recommendation:	It is recommended to configure cookies with the following attributes: • HttpOnly: Prevents access via JavaScript. • Secure: Ensures cookies are only sent over HTTPS. Additionally, review all cookies to ensure sensitive data is not stored unnecessarily.

Finding 6: Information Disclosure via Response Headers	Low
Description:	The application reveals backend technology details through HTTP response headers. Specifically, the "X-Powered-By" header exposes that the application is using Express.js.
Impact:	Exposing underlying technologies can provide attackers with useful information about the system, potentially making it easier to identify known vulnerabilities associated with that technology.
Recommendation:	It is recommended to disable or remove unnecessary headers that disclose system information, such as: "X-Powered-By" This can typically be done through server configuration settings.

Network Exposure Analysis (Nmap Scan)

A network scan was conducted using Nmap to identify publicly exposed services on the target application.

The scan revealed that the application is accessible via:

- Port 3000 (TCP) – **Open**

The service responded with HTTP headers, confirming that a web application is hosted on this port.

The identified open port is expected for application functionality and does not represent a direct security vulnerability.

OWASP ZAP Validation

A passive scan was conducted using OWASP ZAP to validate the findings identified during manual analysis.

The results of the scan were consistent with previously identified issues, including:

- Missing Content Security Policy (CSP).
- Missing HTTP Strict Transport Security (HSTS).
- Insecure cookie configurations.
- Information disclosure through HTTP headers/

This confirms the presence of these misconfigurations and strengthens the reliability of the assessment.

Conclusion

The assessment identified multiple medium-risk security misconfigurations. Addressing these issues will improve system security, data integrity, and overall user trust.